LLM Inference 101 & xLLM

Goals

- Introduce the foundation of LLM inference performance optimization.
- Introduce common optimization strategies and their tradeoffs.
- Introduce xLLM, the general LLM inference framework developed by MLsys

Target Audiences

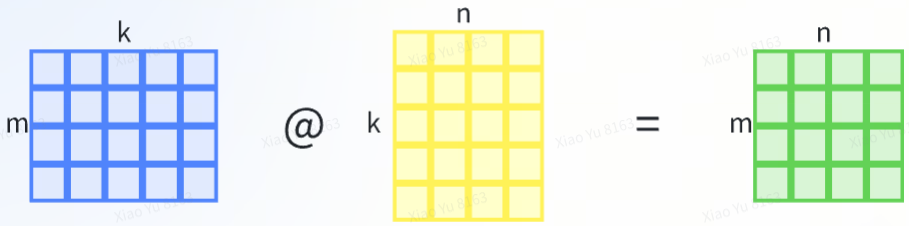
- LLM Product team & LLM MLE

Prerequisite

- Has basic understanding of Transformer, self-attention

LLM Inference Basic

Building Block of LLM: Matmul



- Flops requires: 2 * m * k * n
- Memory bandwidth requires: m*k + k*n + m*n
- Key for LLM performance **flops/mem_bw > flops/mem_bw** in hw spec

example: for A100 bf16, flops/mem_bw = 156.
so [78, 1024] @ [1024, 1024] will have a ok performance
and [12, 1024] @ [1024, 1024] will have a bad performance

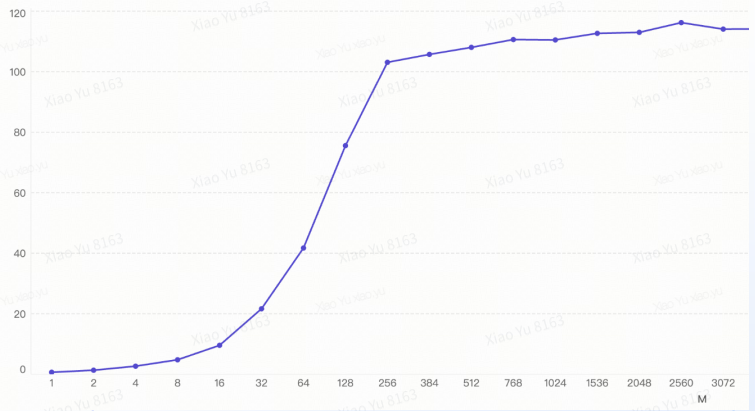
	A100 80GB PCIe	A100 80GB SXM
FP64	9.7 TFLOPS	
FP64 Tensor Core	19.5 TFLOPS	
FP32	19.5 TFLOPS	
Tensor Float 32 (TF32)	156 TFLOPS | 312 TFLOPS*	
BFLOAT16 Tensor Core	312 TFLOPS | 624 TFLOPS*	
FP16 Tensor Core	312 TFLOPS | 624 TFLOPS*	
INT8 Tensor Core	624 TOPS | 1248 TOPS*	
GPU Memory	80GB HBM2e	80GB HBM2e
GPU Memory Bandwidth	1,935 GB/s	2,039 GB/s
Max Thermal Design Power (TDP)	300W	400W ***
Multi-Instance GPU	Up to 7 MIGs @ 10GB	Up to 7 MIGs @ 10GB

MFU

- **MFU (Model Flops Utilization)** as the key performance metric
- MFU v.s. Latency is the key tradeoff in LLM inference

For example:

- Running [78, 1024] @ [1024, 1024] (bf16) on A100 takes **1us**: MFU = **50%**
- Running [12, 1024] @ [1024, 1024] (bf16) on A100 takes **0.7us**: MFU = **10%**



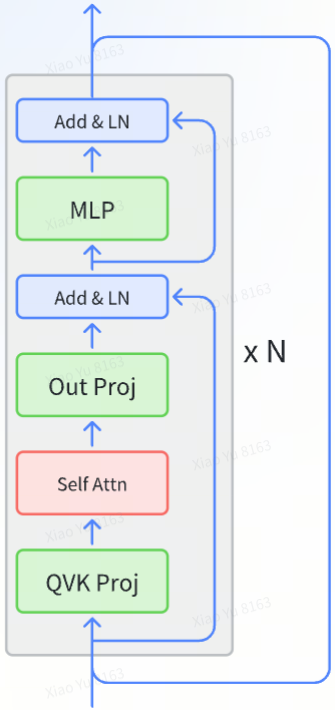
MFU for [M, 4096] @ [4096, 4096] on L20

Goals for LLM inference performance:

Achieve higher MFU without violating product latency SLA

Green box

LLM Perf components: MLP



- Can be simplified as [num_token, h] @ [h, h]
- h*h = dense model weights, e.g.
  - Qwen 2.5 **7B**
  - Llama 3.2 **3B**
  - etc
- Variant: MOE
  - Param: **num_expert***h*h
  - Compute: **top_k***h*h
  - top_k/num_expert: **sparsity** of MOE
  - e.g.
    - Mixtral **39B/141B**
    - Deepseek v3 **37B/671B**

Acheive high MFU in MLP

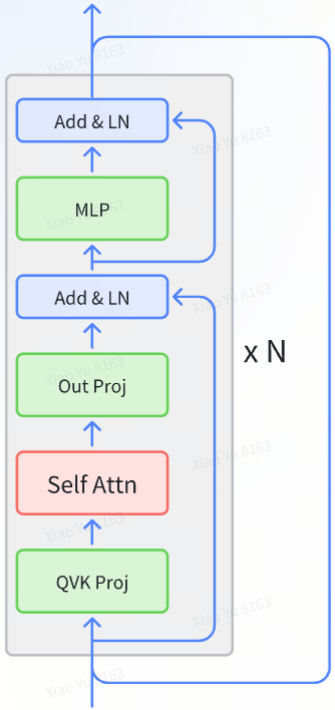
- Dense LLM
  - Flops: 2 * num_token * model_size
  - Mem_bw: model_size
  - So for A100, to acheive high MFU in bf16, **num_token > 78**
- MOE LLM
  - Flops: 2 * num_token * compute_size
  - Mem_bw: param_size
  - So for Deepseek v3 37B/671B on A100, to achieve high MFU in bf16, **num_token > 1414**

	A100 80GB PCIe	A100 80GB SXM
FP64	9.7 TFLOPS	
FP64 Tensor Core	19.5 TFLOPS	
FP32	19.5 TFLOPS	
Tensor Float 32 (TF32)	156 TFLOPS | 312 TFLOPS*	
BFLOAT16 Tensor Core	312 TFLOPS | 624 TFLOPS*	
FP16 Tensor Core	312 TFLOPS | 624 TFLOPS*	
INT8 Tensor Core	624 TOPS | 1248 TOPS*	
GPU Memory	80GB HBM2e	80GB HBM2e
GPU Memory Bandwidth	1,935 GB/s	2,039 GB/s
Max Thermal Design Power (TDP)	300W	400W ***
Multi-Instance GPU	Up to 7 MIGs @ 10GB	Up to 7 MIGs @ 10GB

1. **num_token is the key for MFU in MLP**
2. **It is harder to acheive high MFU in MOE than Dense LLM**

Pink box

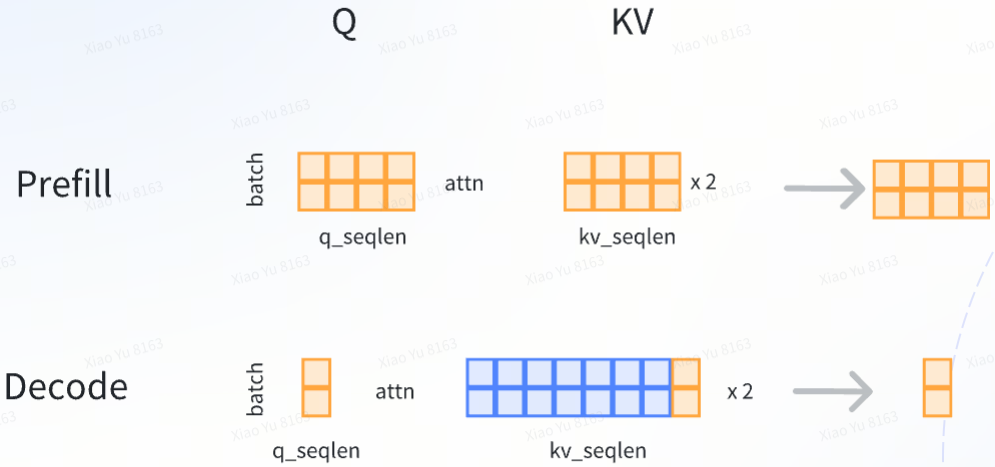
LLM Perf components: Self Attn



$$\text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

softmax([seq, h] @ [h, seq]) @ [seq, h] = [seq, h]

Self Attn: Prefill v.s. Decode



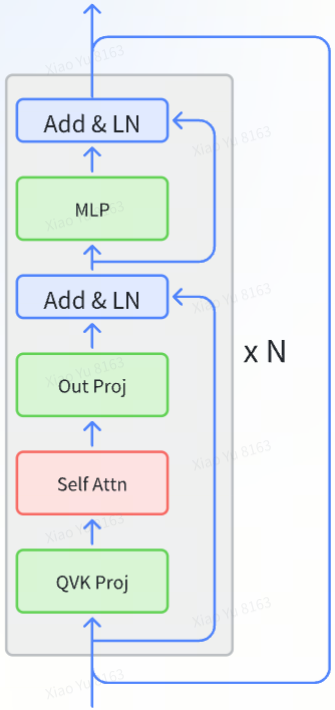
softmax([q_seq, h] @ [h, kv_seq]) @ [kv_seq, h] = [seq, h]

- Prefill: q_seq = kv_seq
  - **Flops/Mem_bw = O(q_seq)**
- Deocde: q_seq = 1, kv_seq = kv_cache + 1
  - **Flops/Mem_bw = O(1)**

"Impossible" to acheive high MFU for self-attn in Decode

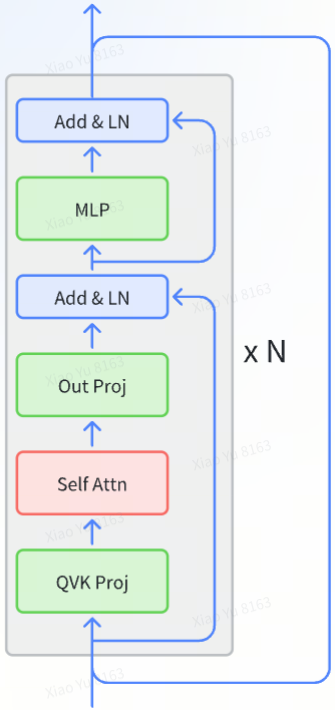
Blue box

LLM Perf components: Other



- Cheap ops
- Key for performance: fuse with other expensive ops

LLM Perf wrap up



- Prefill: num_token = bs * input_length
  - MFU of self-attn is related to input_length
  - MFU of MLP is related to num_token
  - **big input_length = high e2e MFU**
- Decode: num_token = bs
  - MFU of self-attn is always low
  - MFU of MLP is related to num_token
  - **big bs = medium e2e MFU**
- Number of decode steps = output_length

1. **input_length >> output_length: Higher MFU**
2. **input_length << output_length: Lower MFU**

MFU Goals

Using Llama3 training as a reference

GPUs	TP	CP	PP	DP	Seq. Len.	Batch size/DP	Tokens/Batch	TFLOPs/GPU	BF16 MFU
8,192	8	1	16	64	8,192	32	16M	430	43%
16,384	8	1	16	128	8,192	16	16M	400	41%
16,384	8	16	16	8	131,072	16	16M	380	38%

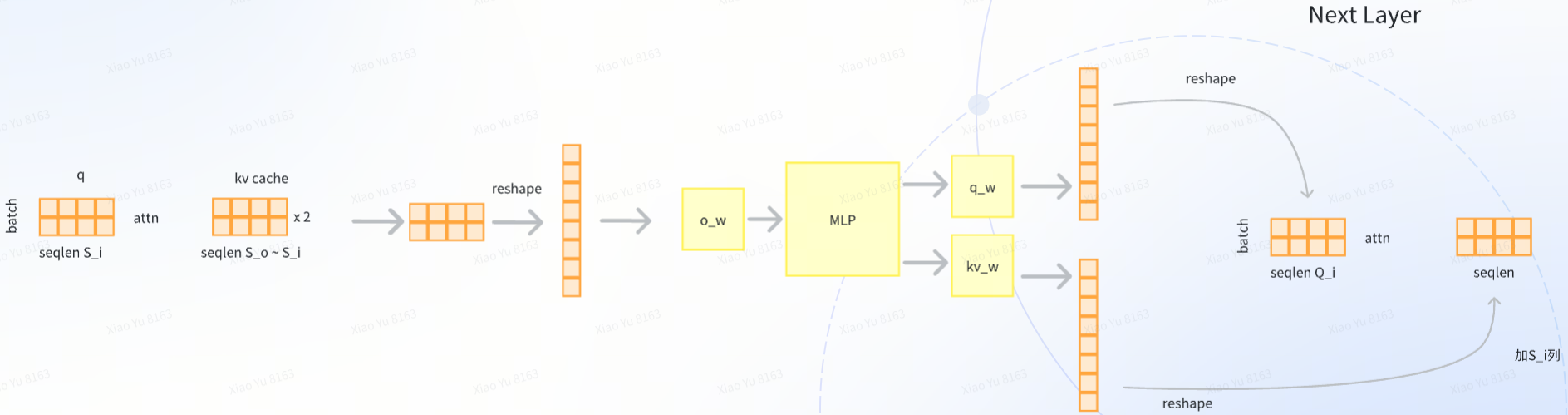
Table 4 Scaling configurations and MFU for each stage of Llama 3 405B pre-training. See text and Figure 5 for descriptions of each type of parallelism.

Note: usually it is easier to acheive higher MFU in training than inference

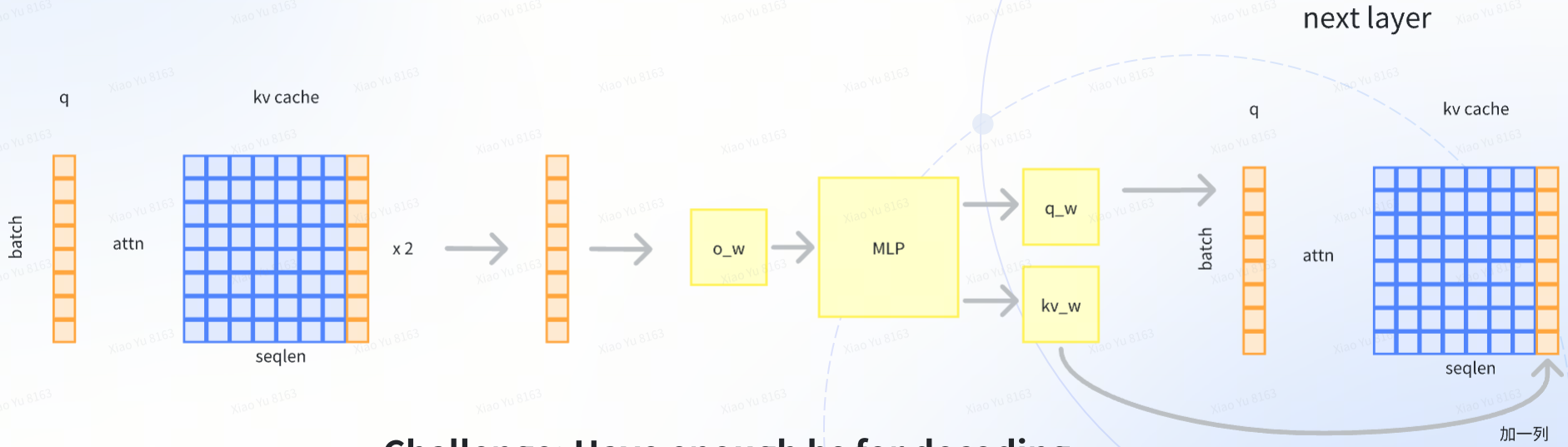
LLM Inference Optimizations

02

Self Attn: Prefill

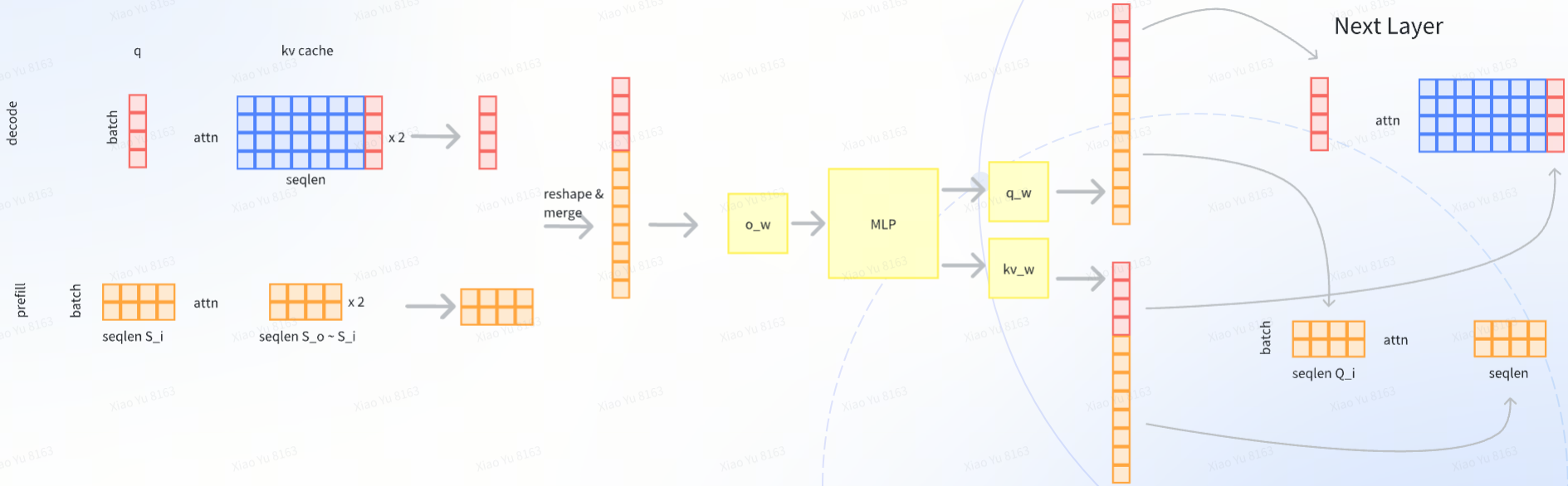


Self Attn: Decode



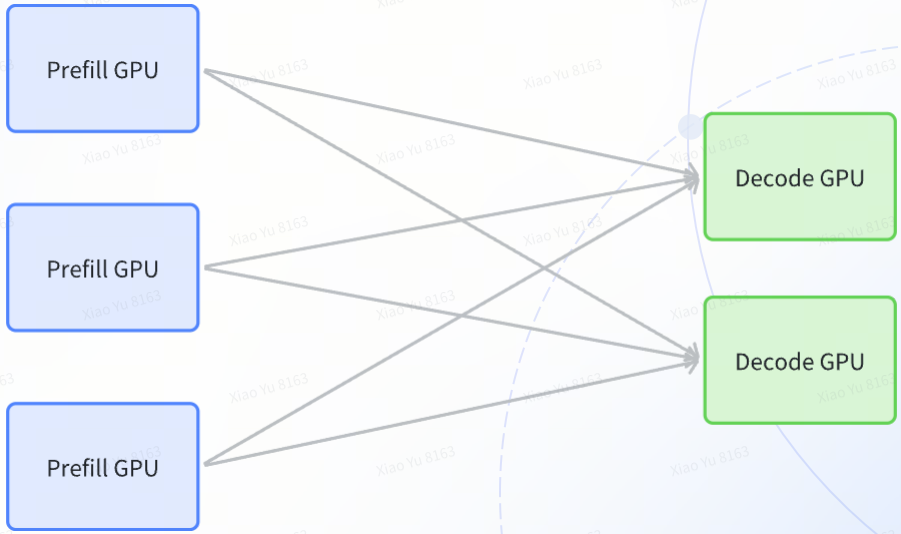
Challenge: Have enough bs for decoding

Batching Strategy #1: PD Fusion



num_token = prefill_bs * prefill_seq + decode_bs

Batching Strategy #2: PD Disaggregation



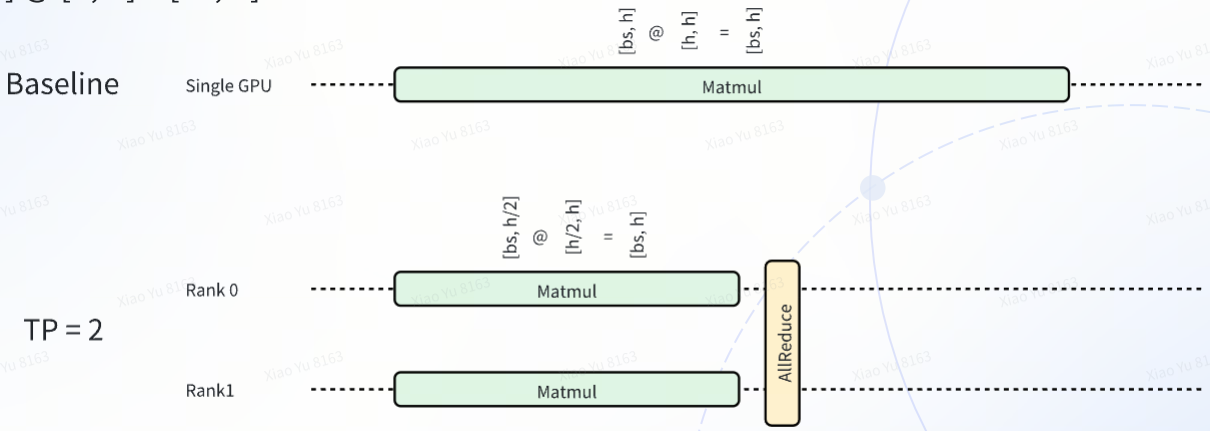
PD Fusion v.s. Disaggregation

	PD Fusion	PD Disaggregation
Pros	• Easies to deploy • Less concurrent requests in one instance. Better for failure recovery. • Easier to acheive peak MFU in one instance	• Can acheive faster latency • Heterogeneous hardware support
Cons	• Usually higher latency (TTFT & TPOT) since share computation resouce for prefill & decode.	• Way more challenging for deployment • Hard to manage Prefill & decode ratio.

Peak MFU are similar between these two strategies

Model Parallelism

Example: [bs, h] @ [h, h] = [bs, h]



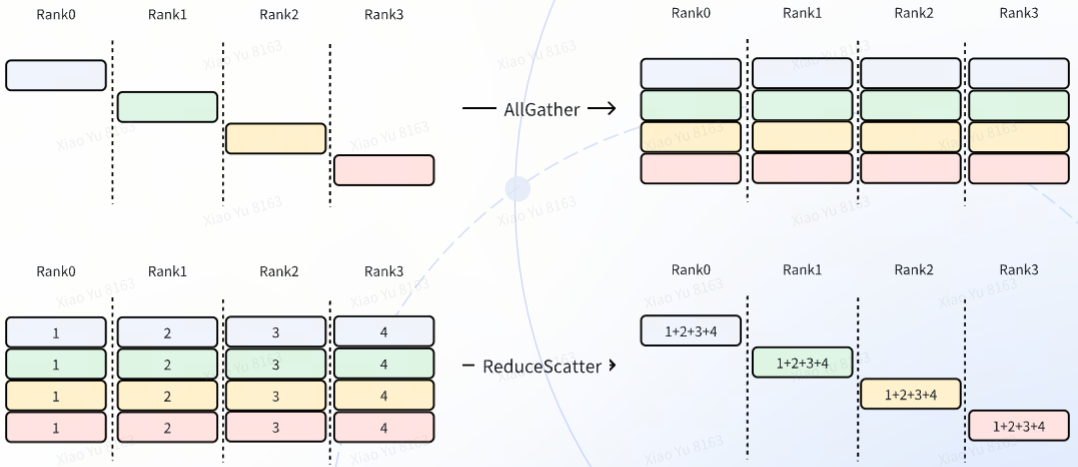
- Largely reduce latency by parallel computing on multiple GPU
- Pay an extra cost for data communication across GPU

1. **Fit a larger model or support a longer seq**
2. **Improve latency by sacrificing MFU**

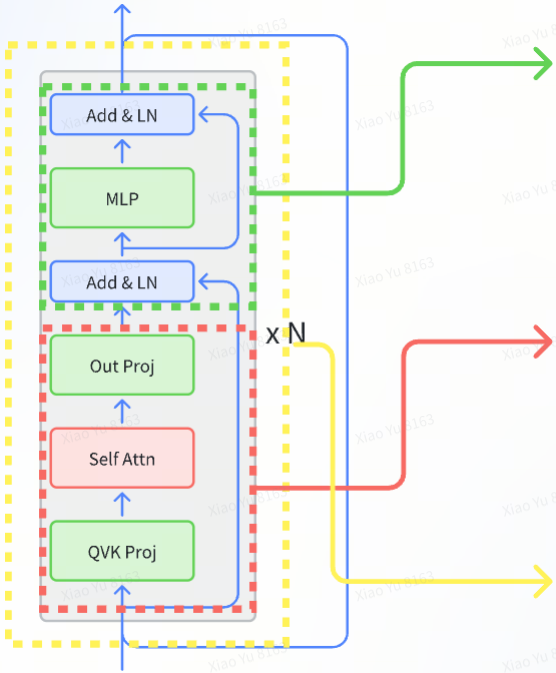
Data Communication Operations

Different ops have:

- different data type requirements
- communication efficientcy



Model Parallelism Options



MLP Block
- TP (TensorParallel): AG + RS all activations
- EP (ExpertParallel, MOE only): A2A expert activations

Attention Block
- TP (TensorParallel): AG + RS all activation
- SP (SequenceParallel): AG full sequence activation
- SP-Ulysses (SP Variant): A2A attention head
- DP (DataParallel): N/A
- CP (ContextParallel): AG full sequence kv

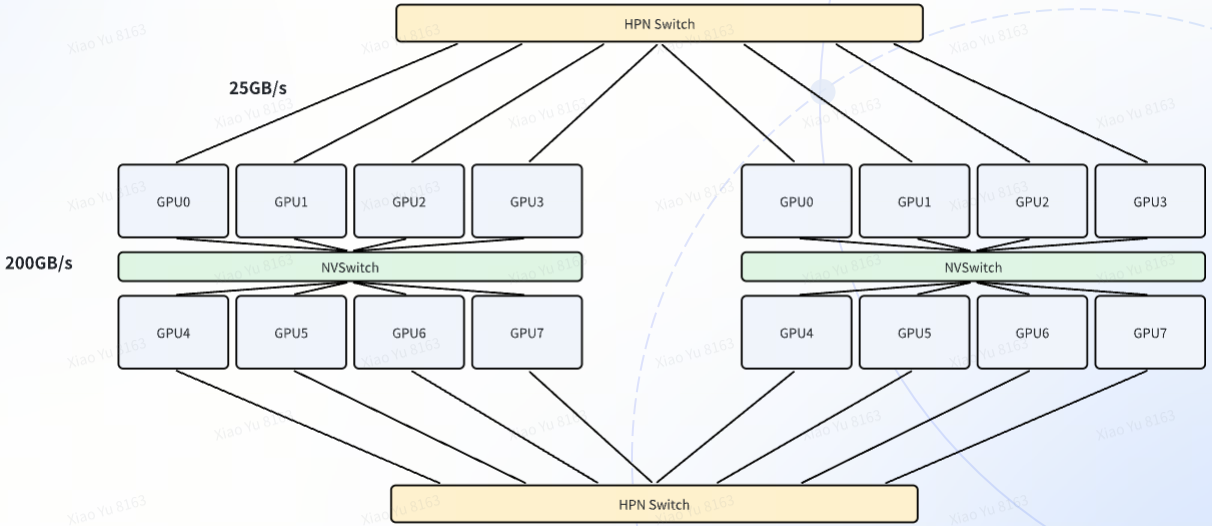
Transformer Block
- PP (PipelineParallel): Send-recv all activations

xD Model Parallelism

- We can mix-and-match multiple model parallel strategy. -> multi-dimension parallel
- Real world examples:
  - Most common case: Attn TP8, MLP TP8
  - Doubao
    - Attn DP2TP4, MLP EP2TP4
    - Attn SP8, MLP TP8
    - Attn DP4TP8, MLP EP4TP8
    - Transformer PP8, Attn SP8
  - DeepseekV3 paper: Attn DP80TP4, MLP EP320

Communication Channel

Different channels have different network bandwidth

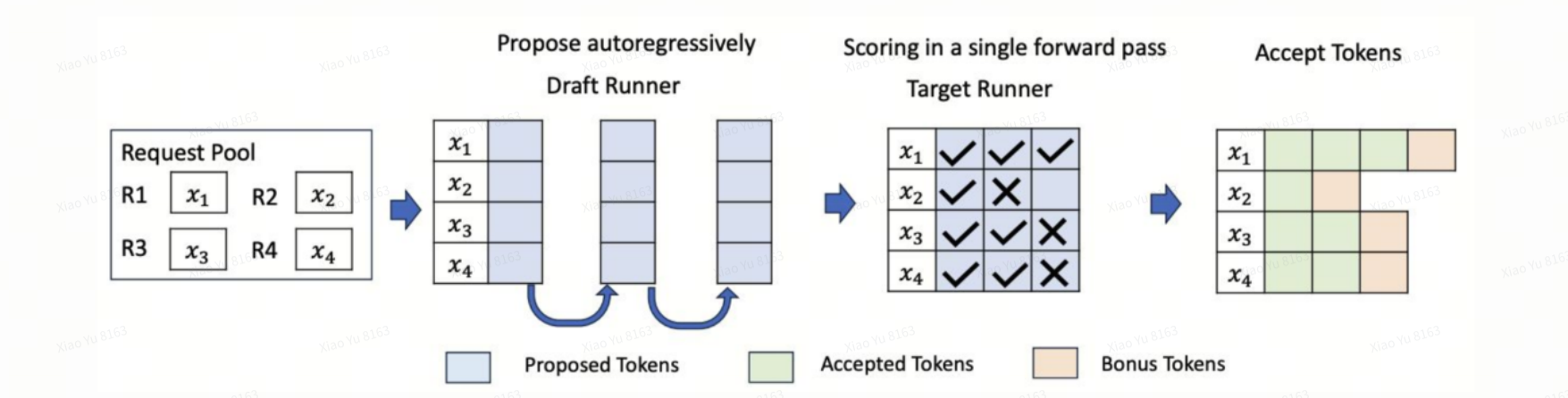


"Which parallel dimension to use which channel" is critical

Hardware Options

	A100	H20	L20	H100	Lianhuashan
Int8 tflops	624	296	239	1979	752
FP16/BF16 tflops	312	148	119.5	989	376
Mem bw (GB/s)	2000	4000	864	4000	1200
Mem capacity (GB)	80	96	48	80	64
Node size	8	8	1	8	16
Inter node bandwidth (GB/s)	200	450	n/a	450	196
Intra node bandwidth (GB/s)	25	25	25	25	25
Cost Ratio	1	1.1	0.5	3.5	1

Speculative decoding



Improve TPOT by sacrificing MFU

Speculative Decoding

Use an example to understand its tradeoff!

- Baseline: **TPOT: 50ms, MFU 40%**
- Speculative decoding setup:
  - Draft: 2 tokens
  - Accept rate: 60%
- Expected accept tokens in each run: 1 + 0.6 + 0.6^2 = 1.96
- **TPOT** with speculative decoding: 50ms / 1.96 = **25.5ms**
- Effective **MFU** with speculative decoding: 40% * 1.96 / 3 = **26.1%**

Speculative Decoding

Case study on qwen 2.5 7B, PD fusion, input:output=2000:100

	¥/M token	
	no-spec	spec
H20	0.12934	0.13195
A100	0.05037	0.04895
L20	0.05416	0.05663
L40	0.04206	0.04206
Lianhuashan	0.04909	0.046667
Ada1	0.056103	0.04586
Trillium TPU	0.032204	0.02812

Both input_length:output_length & HW spec play a critical role for speculative decoding !!

Other Optimizations

- Lossly Optimizations
  - Quantization (weight, activation, context)
    - **w8a8c16 (Most common)**
    - w4a16c16
    - w8a8c8
  - Sparsity
    - Sparsity Attention
    - MLP sparsity
  - Skip Layers
  - Low rank
    - Deepseek MLA
- Session/prefix kv-cache
- Reuse kv
  - RadixAttention
- etc...

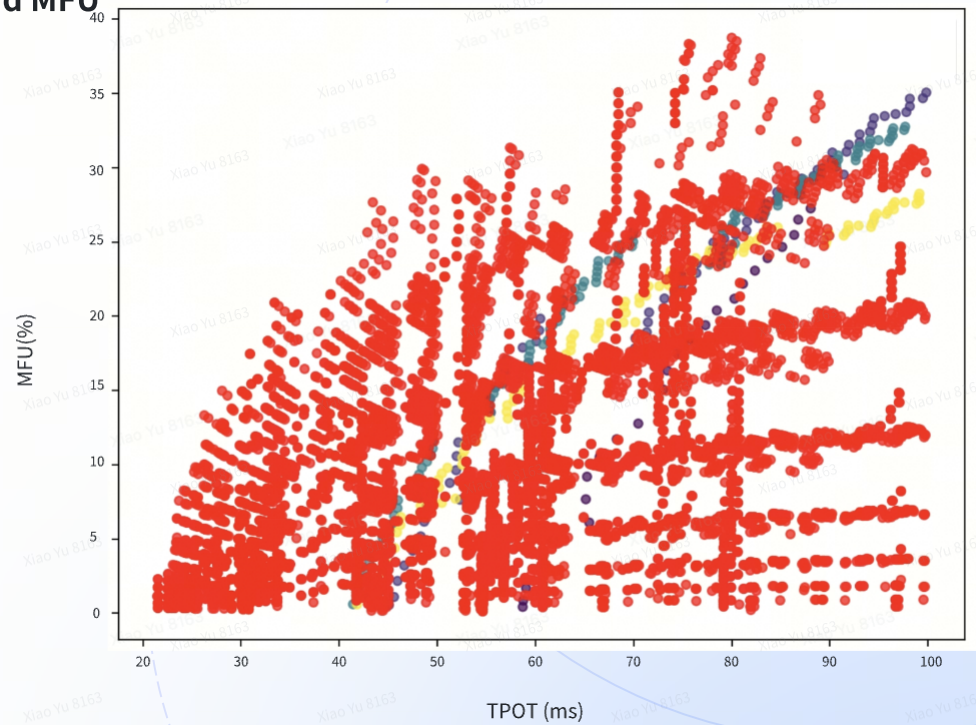
Put everything togather

LLM Performance is a tradeoff between **Latency and MFU**

Critical Factors:

- Input length/output length
- TTFT & TPOT SLA
- Hardware Options
- Model arch: Dense v.s. MOE
- Model size
- Batching strategy:
  - PD fusion v.s. PD disaggregation
  - num_token, prefill_seq, bs
- xD Model parallelism
  - DP, TP, SP, EP, PP, etc...
  - Map parallel strategy to network channel
- Speculative decoding or not
- Lossly optimization

A unoptimized configuration can cost 2x,3x or even more extra HW cost!!



xLLM

03

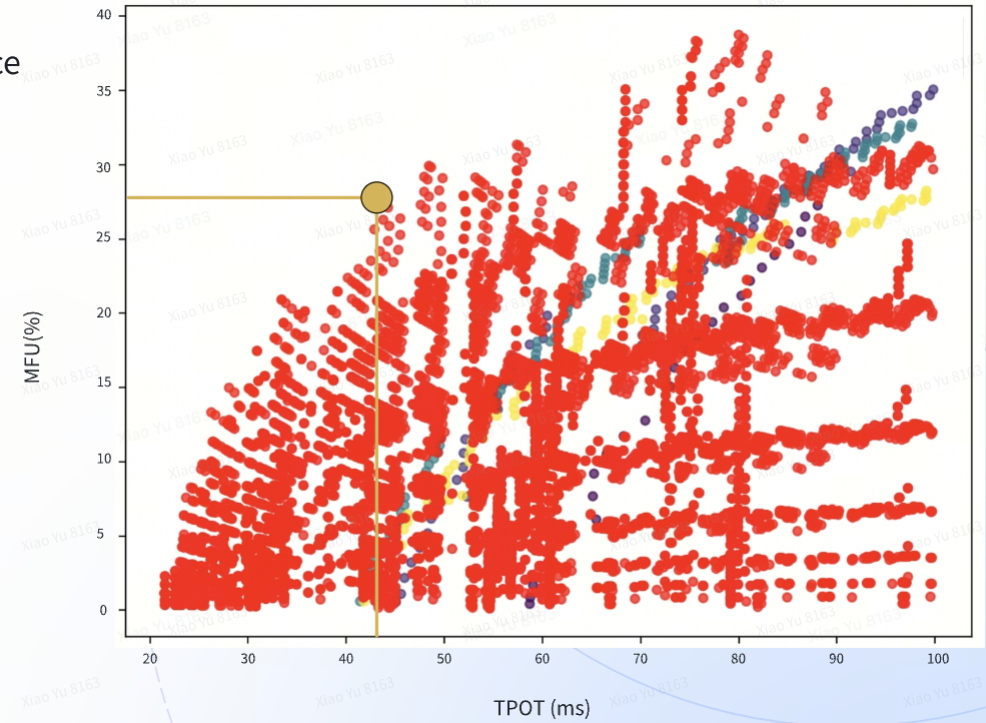
Let's introduce: 🎉xLLM!!! 🎉

xLLM: All-in-one LLM inference engine from MLSys

Verified on Doubao, **now available** for all ByteDance LLM products! 🎉

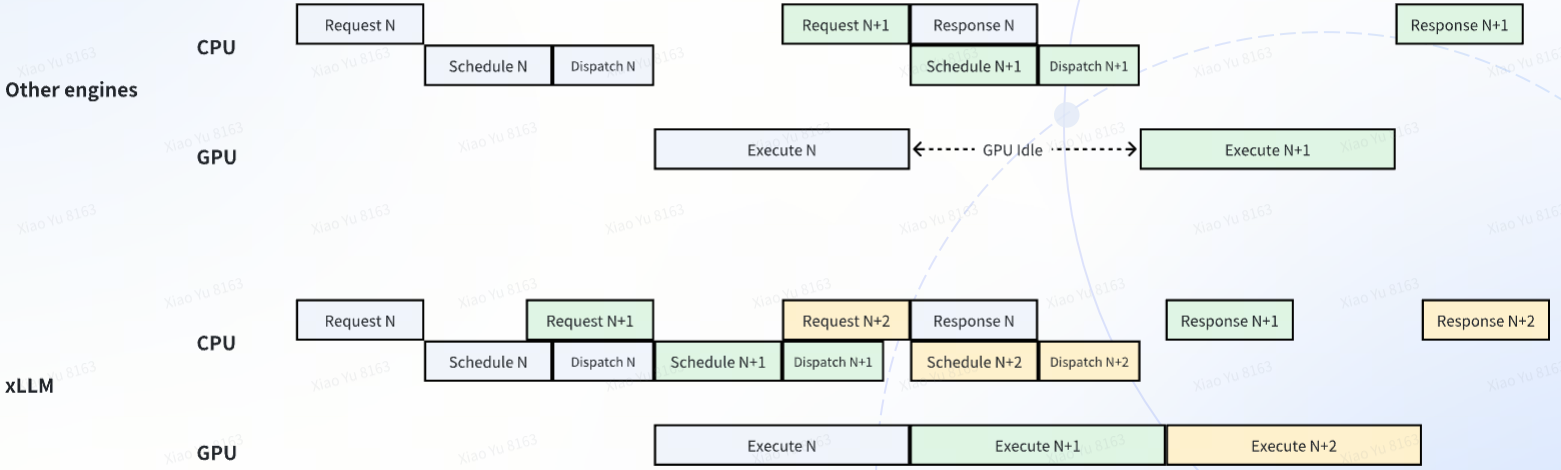
What we need:
- Model checkpoint + Model arch
- Latency SLA (TTFT, TPOT)
- Resource pool

What we can provide:
- Deployment with optimal config (**best MFU**)
- More cost effective suggestions, e.g.
  - Loose latency requirement
  - Alternative HW choice
  - lossly optimization



Extra optimizations in xLLM

Real async inference

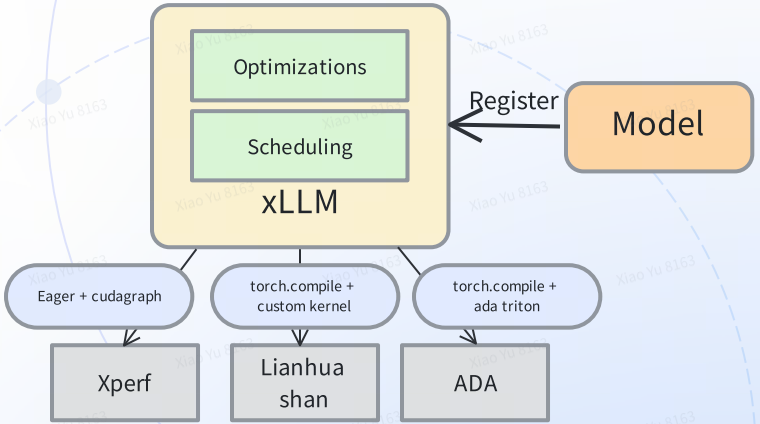


Can acheive 100% GPU utilization on production

Extra optimizations in xLLM

Heterogeneous HW support with in-house optimized kernel

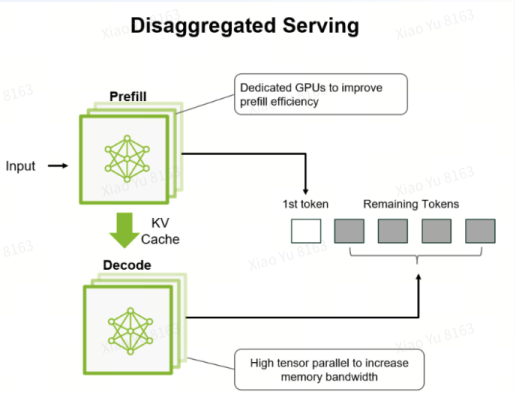
- Supports all HW in ByteDance data center
  - NV GPU, NPU, MLU, ADA, more to come
- GPU kernels
  - XPerf kernels
  - Communication overlapping
    - Flux: Fast Software-based Communication Overlap on GPUs through Kernel Fusion https://arxiv.org/html/2406.06858v1
    - Comet: Fine-grained Computation-communication Overlapping for Mixture-of-Experts https://arxiv.org/html/2502.19811v3
- Lianhuashan kernels
  - xpu_ops
- Ada (WIP)



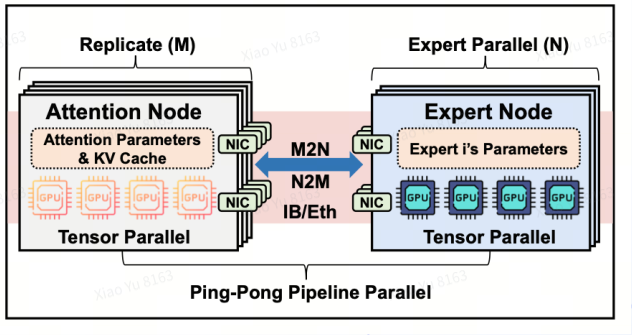
Extra optimizations in xLLM

Multi-host inference

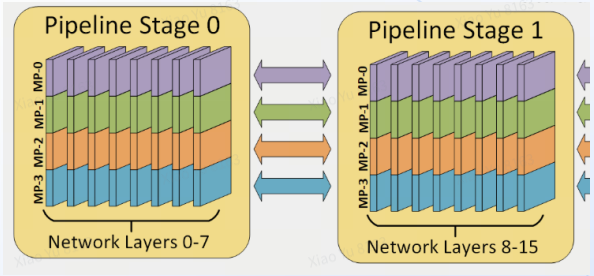
- Bundle with Merlin Scheduling System to enable multi-host inference.



Multi-host PD Disaggregation



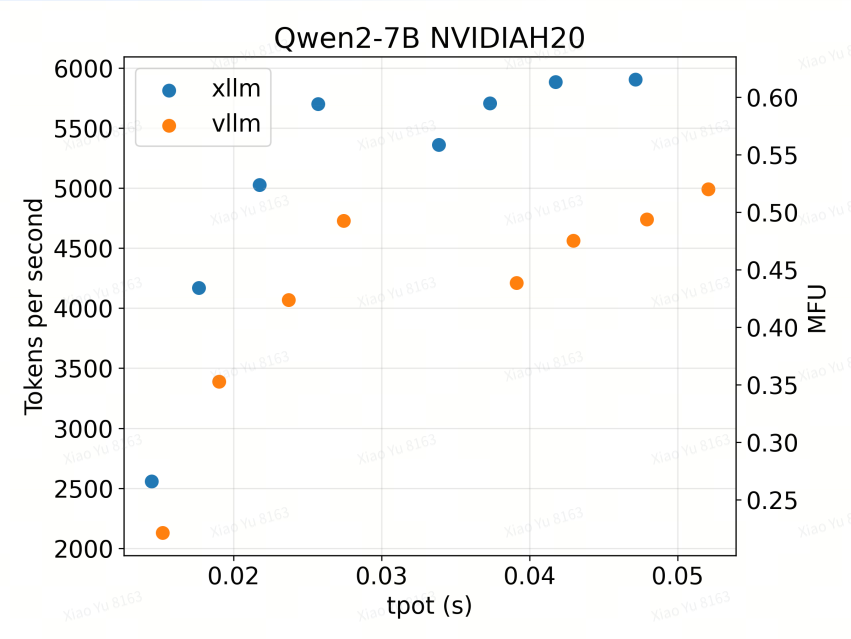
Multi-host EP
Supported for Seed Models

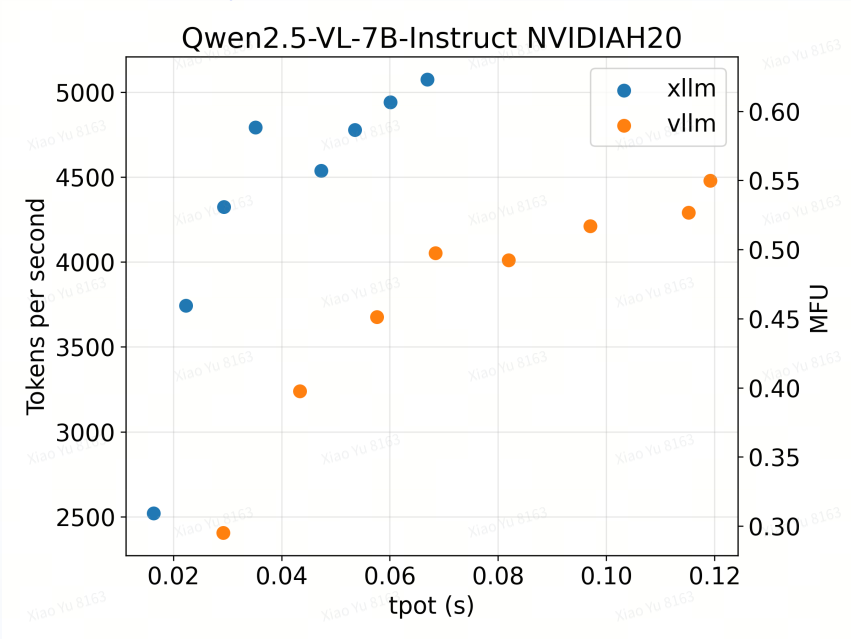


Multi-host Zero-Bubble Pipeline Parallelism

- https://ieeexplore.ieee.org/document/10609649
- https://arxiv.org/abs/2504.02263

xLLM v.s. vLLM

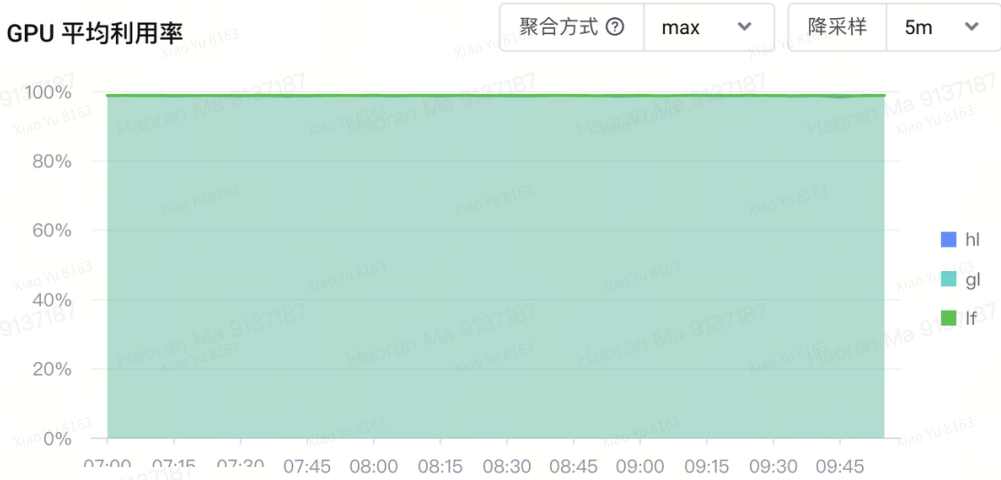


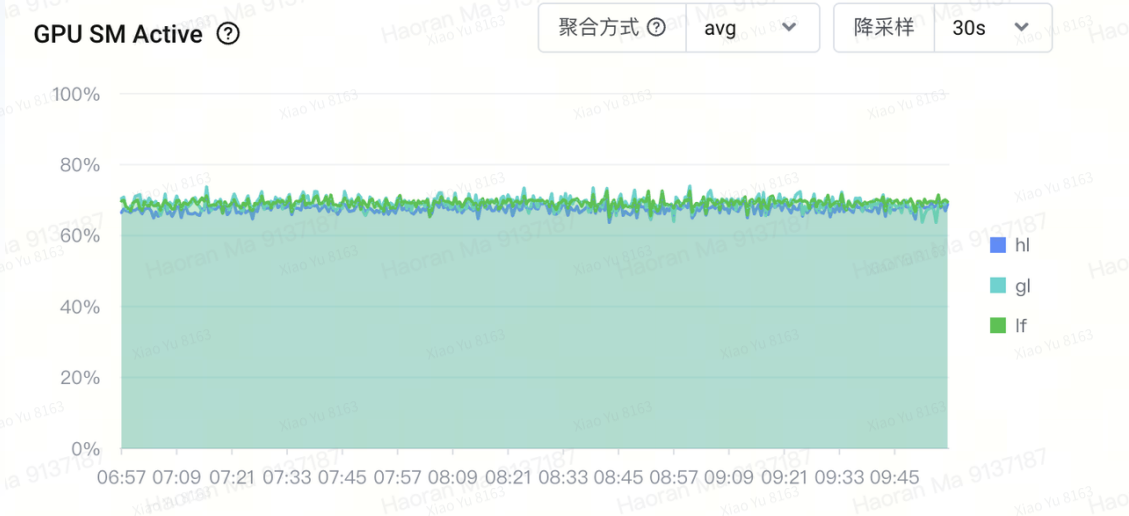


vllm version:0.8.4

xLLM load test

GPU util and SM active for xLLM





Enable your team to acheive company's GPU efficiency goal

xLLM Production Example

Ads Security VLM on production dataset

Datatype	Hardware	baseline QPS	xLLM QPS	Improvement
fp16	Lianhuashan	0.9	1.24	+35%
fp16	L40	0.6	0.81	+35%

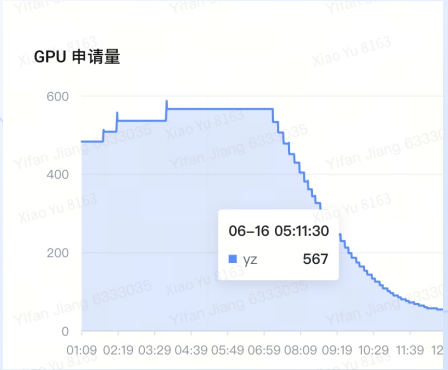
xLLM Production Example

UniVL-2B

Framework	Hardware	QPS Per Card	Total Peak QPS	Total Number of L20s Needed
xLLM	L20	8.53	2150	252
LMDeploy	L20	3.79	2150	567

xLLM:

LMDeploy:



xLLM Production Example

More Success Stories and User Feedback...

InternVL 68B, TP, Batch Processing:

金辉

这里以8卡为例哈，caption任务，100张图片。使用xllm之前总共需要耗费11min23s(只能单实例部署), 使用xllm fp16(单实例部署)需要耗费4min29s， 使用xllm fp8(双实例部署)需要耗费2min53s。

TikTok Search & Recommendation, Thoth Model Online Serving:
User's Performance Comparison Against Other Frameworks

- 依据现有压测效果，XLLM框架 + VLLM 优化效果更优
  - 相比较与Laplace - Python 3.9 + XPERF优化，吞吐可以提升至3倍

xLLM User Experience

xLLM ❤️ Merlin

- xLLM & Merlin are one team that commits to deliver best in-house inference experience
- Two focuses:
  - **Online inference**
    - Product SLA guarentee, including latency & reliability.
  - **Offline inference**
    - Throughput, scale & flexibility

xLLM User Guide

- Use xLLM as official inference image on Merlin
  - Online inference: xLLM推理框架使用教程
  - Offline inference: xLLM离线任务教程
- Submit your requirements
  - 🚀 xLLM 推理框架提需模板文档

xLLM User Group

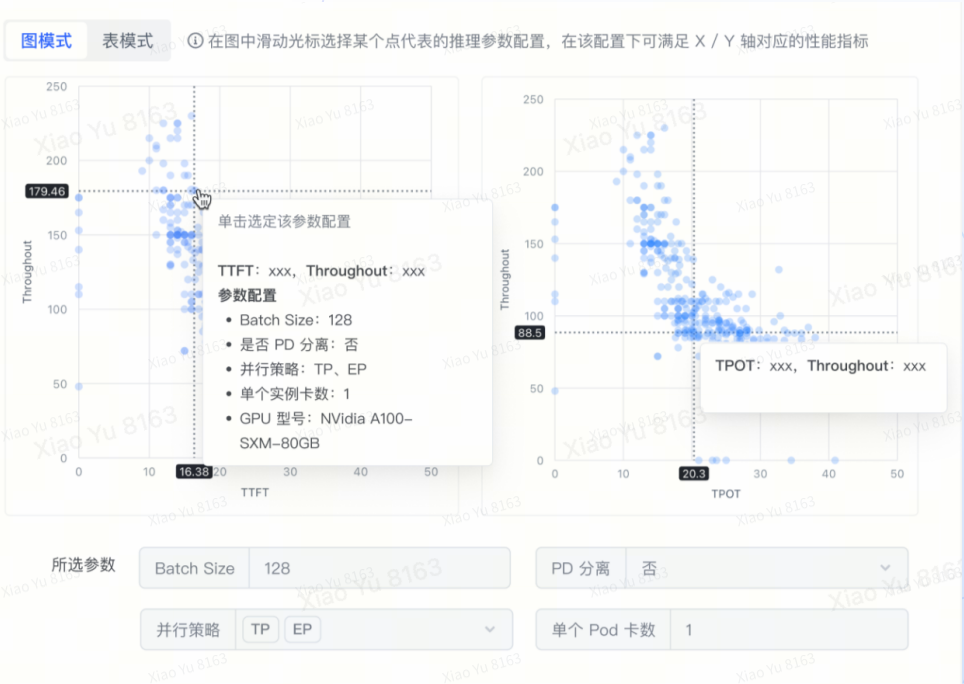
ByteDance

Only members of ByteDance can join this group.

This QR code is valid permanently

What is coming next?

- Q3
  - Streamlined LLM inference experience on Merlin
    - Allow selecting MFU/TTFT/TPOT explicitly
  - Tool calling support
  - Accelerate RL using xLLM
  - More model+hw support



Q & A